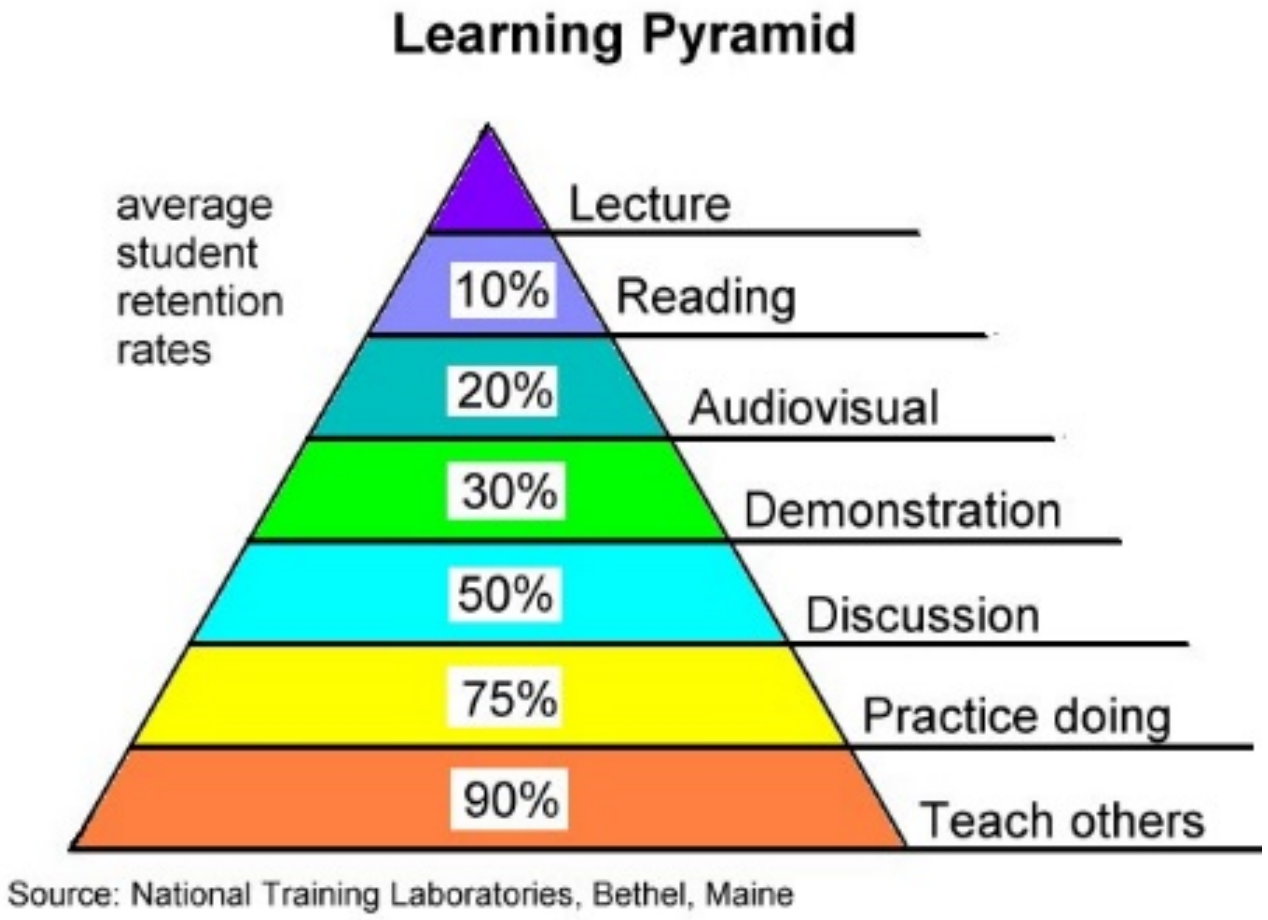
OOAD – Class Diagrams, etc.

R.S. Madanayake

The Learning Theory Pyramid



One of my most admired lecturers …

- Dr. Ruvan Weerasinghe said,

“The less I teach the more you learn.”

This Does NOT mean I will NOT teach you anything.

I just means I will spoon feed you less and encourage you to learn on your own more (Hopefully !!!!!!!).

Agile Software Development Methods

- This is **NOT** UML, but a software development method that is newer than UML.
- Do NOT forget UML, because many Agile projects still use UML (mainly class diagrams and use case diagrams).
- Under AGILE specifically we will deal mainly with User Stories, Epics and CRC cards.
- Agile Software Development was **said to be** mainly Customer/Client Oriented (by its founders) as opposed to the Waterfall life cycle which is more Documentation Oriented.

User Story

Simpler and shorter way of describing a scenario from a user perspective. Popular with Agile Methods

Format:

As a <type of user>

I want <goal>

So that <Reason>

Eg. As a Bank Customer

I want to change my PIN online

So that I do not want to go the Bank

A team at Connextra developed the traditional user-story template in 2001

Expresses something that
a user of the system wants to be able to do.

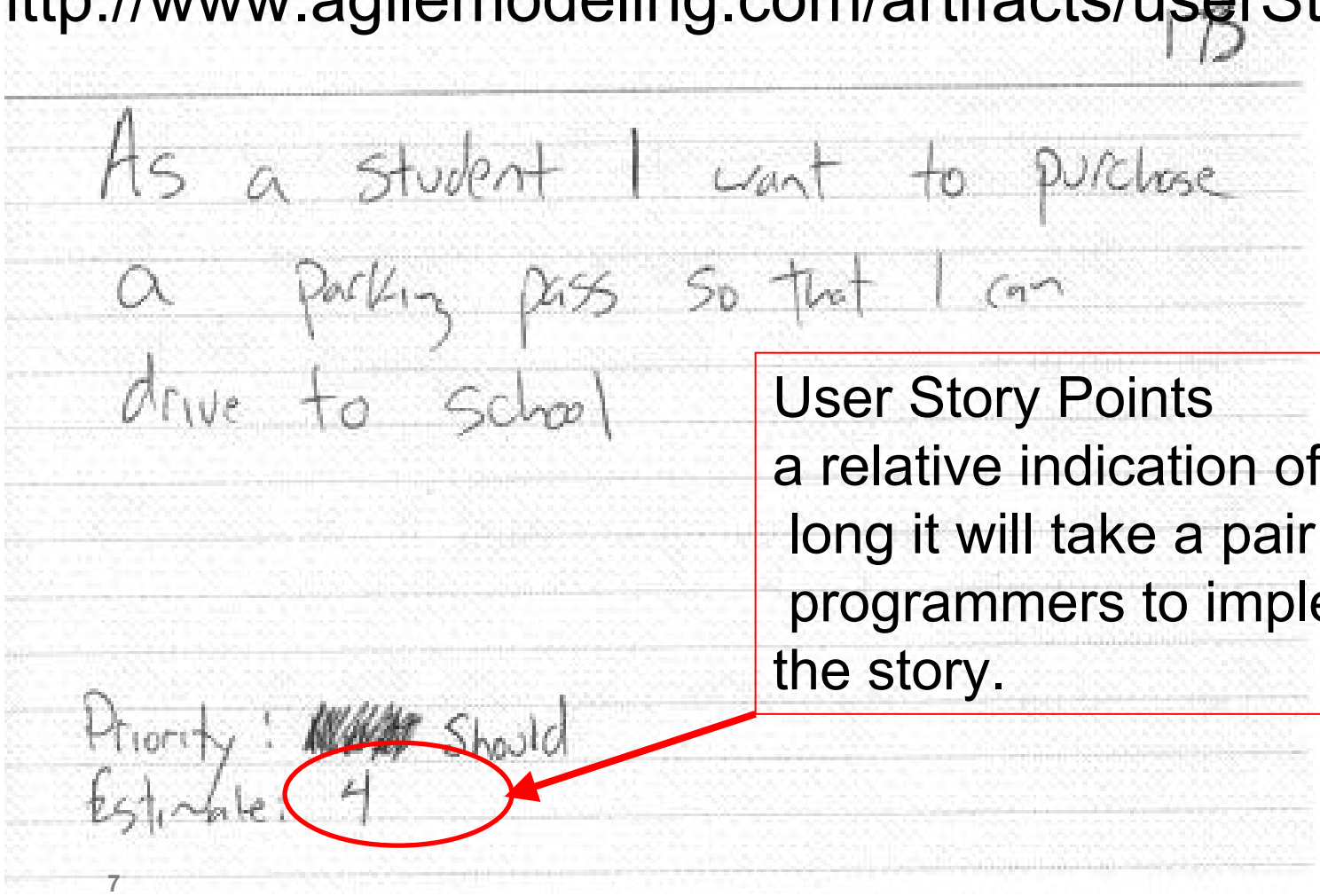
In class Exercise

- Example user stories: -
  - **As a** Businessman **I want to** View Sales **so that I can** Plan my business.
  - **As a** Scientist **I want to** Analyse Data **so that I can** Finish my experiment.

- The 2 above are user stories from 2 extremely different scenarios.
- Your task is to write down 10 **other** user stories **in addition to the above**.

User Story Card (Front)

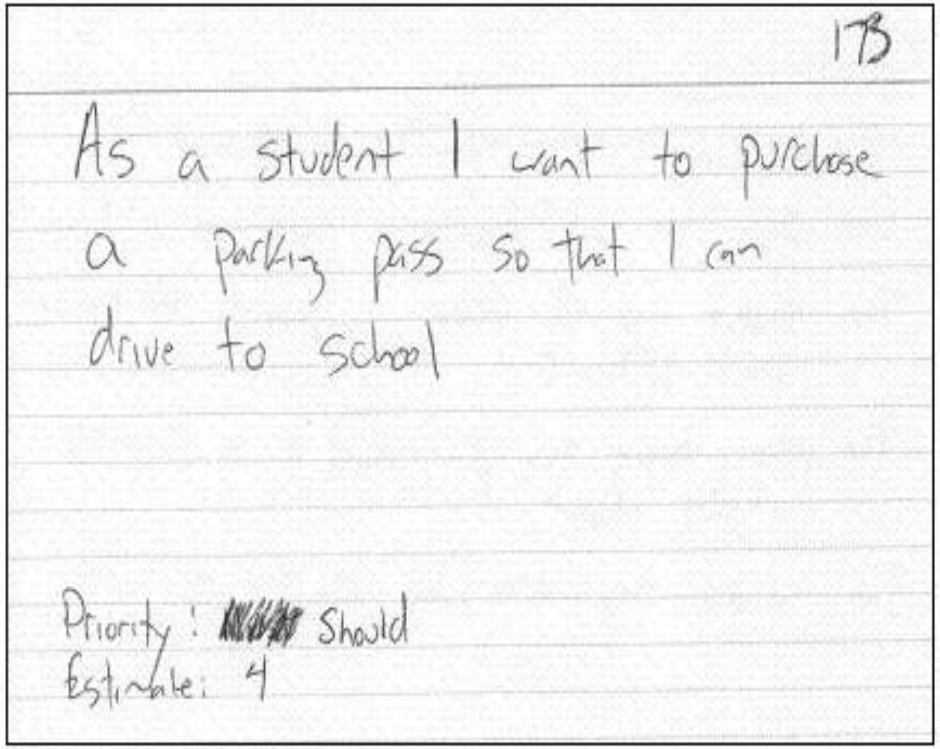
http://www.agilemodeling.com/artifacts/userStory.htm



User Story Points
a relative indication of how long it will take a pair of programmers to implement the story.

User Story Card (Back)

Front of Card

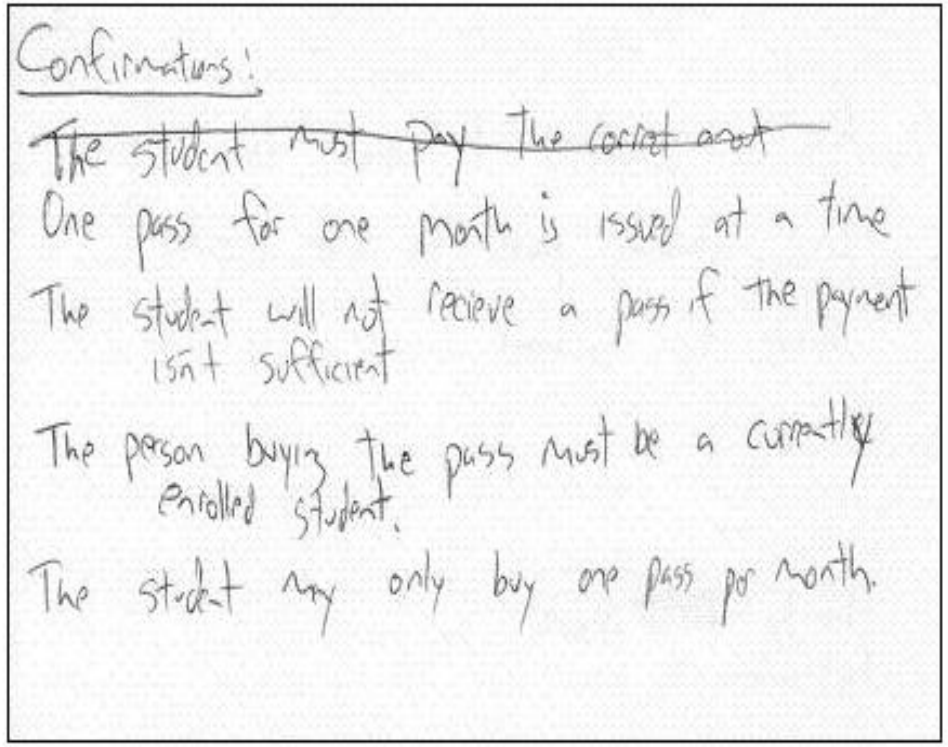


1B

As a student I want to purchase a parking pass so that I can drive to school

Priority: Should
Estimate: 4

Back of Card



Confirmations:

~~The student must pay the correct amount~~

One pass for one month is issued at a time

The student will not recieve a pass if the payment isn't sufficient

The person buying the pass must be a currently enrolled student.

The student may only buy one pass per month.

Use Story

As a (late riser), I want (to get my breakfast quickly)

in order (to catch the train)

- **Can break this into number of more detailed stories about real requirements**

As a (late riser), I want (a four-slice toaster)

in order (to make breakfast quicker)

And

I want (a folding bike) in order (to get to the station quicker)

What are Epics, Themes?

User Stories used in Agile Development

Epics

- Large User Stories which CANNOT be completed in one **sprint.**
- **What is a sprint?**

- An Epic maybe decomposed into many smaller user stories.

As a salesman I want to do sales so that …….

- **Can be decomposed into;**

As a salesman I want to attract customers

As a salesman I want to do transactions

As a salesman I want to print invoices

As a salesman I want to accept payments

Themes

- Many smaller user stories organised around a common theme.
- (But they are NOT part of a larger user story – as opposed to Epics)

Two types of UML Diagrams

- Structural Diagrams
- Behavioural Diagrams

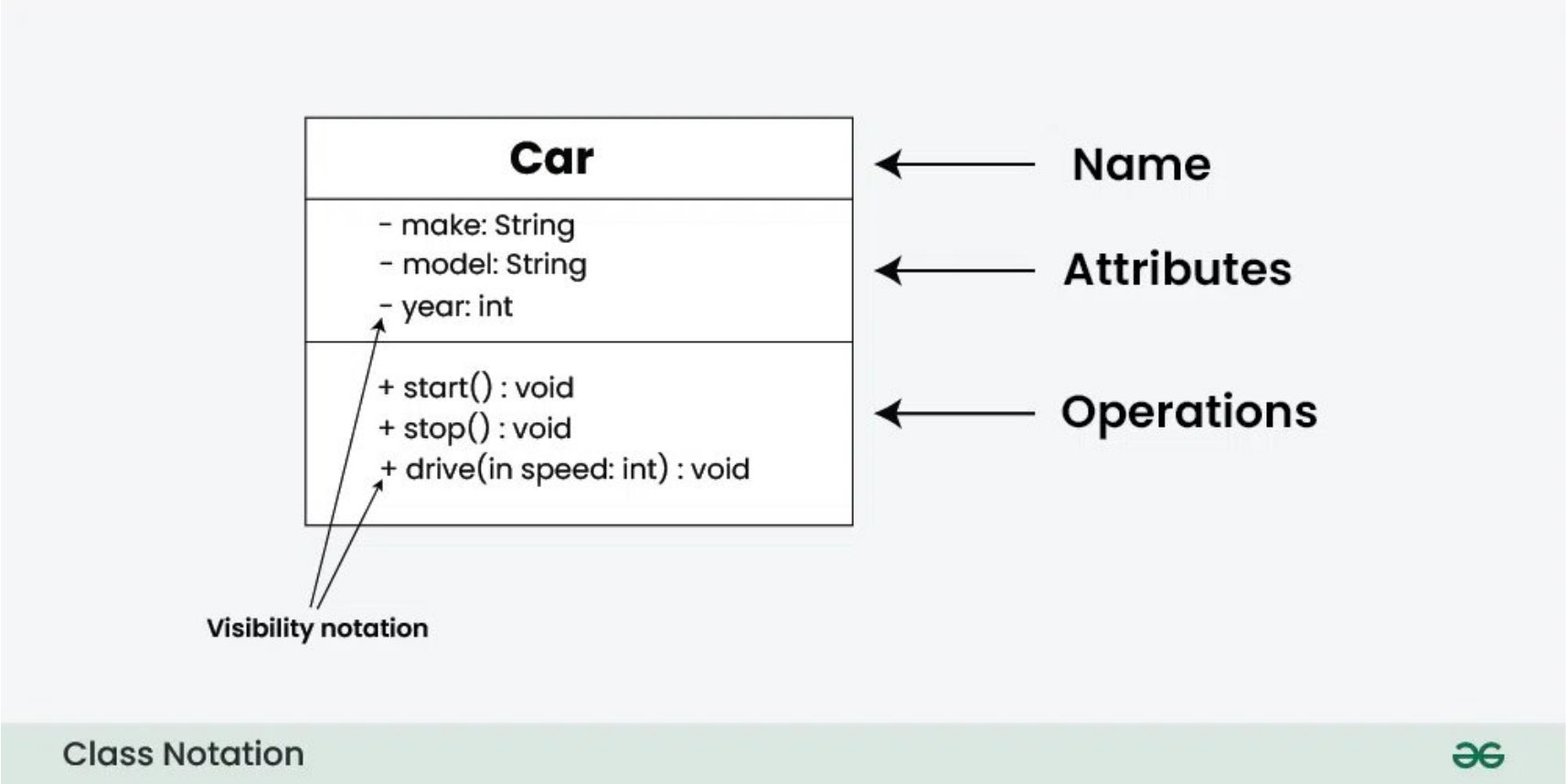
Structural UML Diagrams / Structure Diagrams

Structure diagrams emphasize the structure of the system – using objects, classifiers, relationships, attributes and operations. They are used to document software architecture.

- Class diagram
- Component diagram
- Composite structure diagram
- Deployment diagram
- Object diagram
- Package diagram
- Profile diagram

A good example is the class diagram

- One single class in a class diagram looks like this : -



Objects can be created from a class

- Eg. Class (dog) => Object (Bobby)

- **Objects** created from a **class** are called **instances of that** (respective) **class**.

Class Name

- The name of the class is typically written in the top compartment of the class box and is centered and bold.
- It is the most compulsory information in a class.

Attributes

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

Methods

- Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

Visibility Notation:

Visibility notations indicate the access level of attributes and methods. (Compare your house, belongings, relatives/friends and strangers)

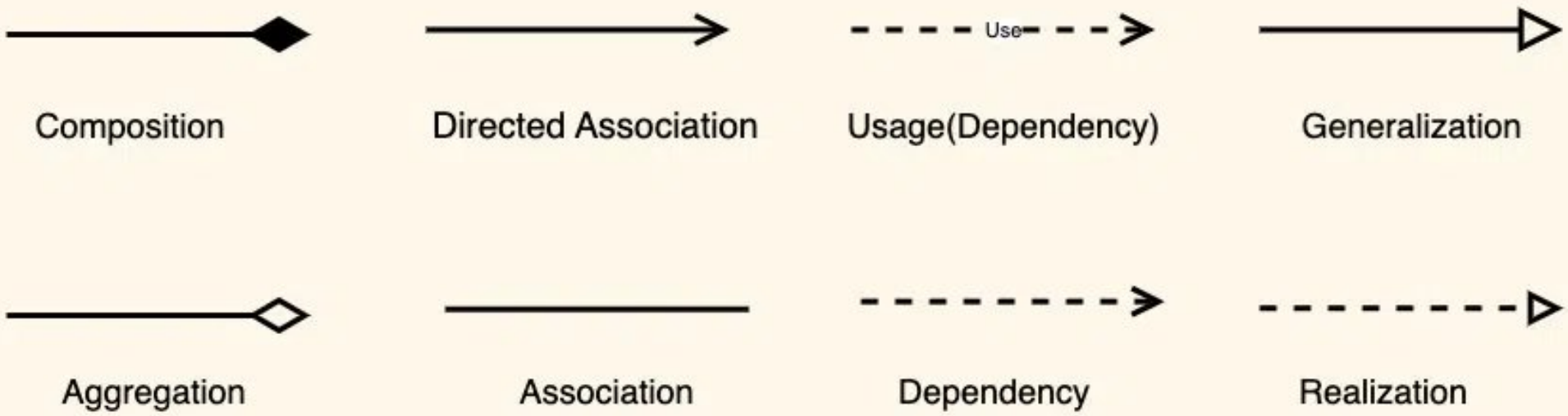
Common visibility notations include:

- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

A Class Diagram

- A class diagram usually has more than one class.
- In that case, there are relationships between those classes.

Class Diagram Relationships



Association

- An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.

Directed Association

- A directed association in a UML class diagram represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

Aggregation

- Aggregation is a specialized form of association that represents a "**whole-part**" relationship. It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part). Aggregation is represented by a diamond shape on the side of the whole class. In this kind of relationship, the **child class can exist independently of its parent** class.
- eg. In a Computer Store which sells both “whole” computers as well as computer parts. Here, the parts (child classes) can exist independently of the computers (parent classes).

Composition

- Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship. In composition, the part class cannot exist independently of the whole class. Composition is represented by a filled diamond shape on the side of the whole class.
- Eg. In a car sales company which does NOT sell car parts;

Headlight **<u>is part of</u>** car

(Compare Composition with Aggregation)

Generalization (Inheritance)

- Inheritance represents an "is-a" relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent). Inheritance is depicted by a solid line with a closed, hollow arrowhead pointing from the subclass to the superclass.
- eg. Cat (child/subclass) “**is a**” Mammal (parent/superclass)

Realization (Interface Implementation)

- Realization indicates that a class implements the features of an interface. It is often used in cases where a class realizes the operations defined by an interface. Realization is depicted by a dashed line with an open arrowhead pointing from the implementing class to the interface.

Dependency Relationship

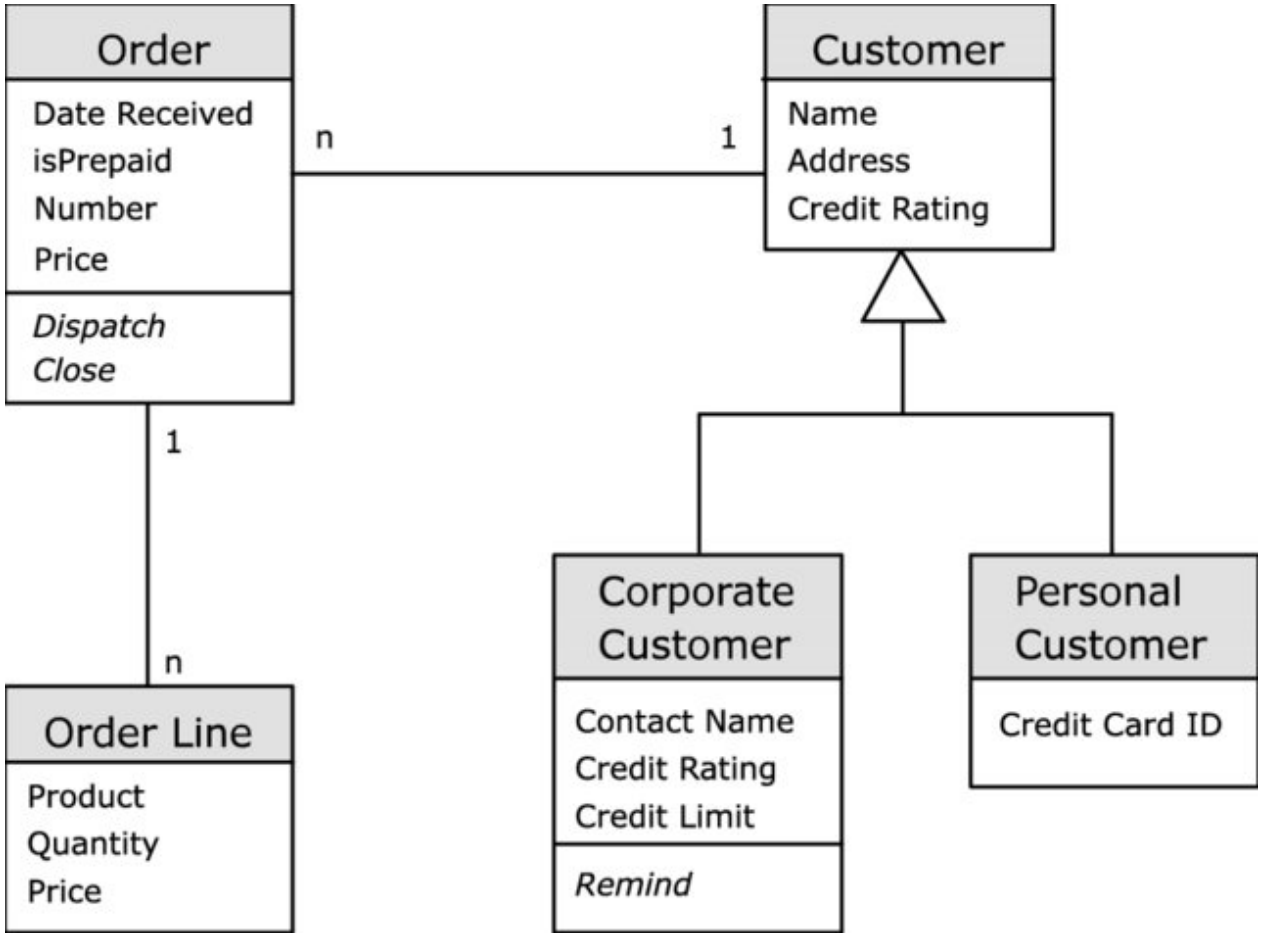
- A dependency exists between two classes when one class relies on another, but the relationship is not as strong as association or inheritance. It represents a more loosely coupled connection between classes.

Usage (Dependency) Relationship

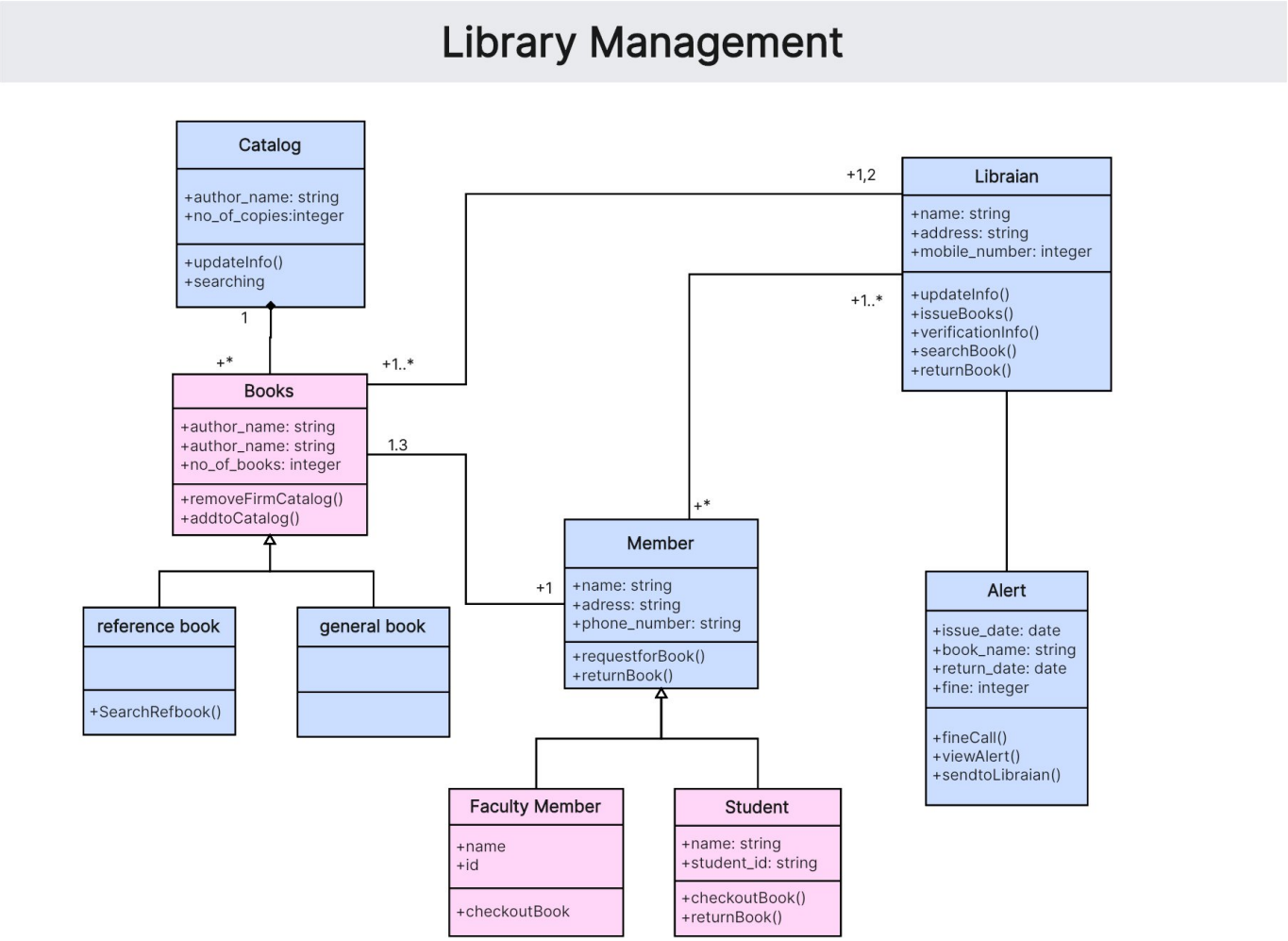
A usage dependency relationship in a UML class diagram indicates that one class (the client) utilizes or depends on another class (the supplier) to perform certain tasks or access certain functionality. The client class relies on the services provided by the supplier class but does not own or create instances of it.

- In UML class diagrams, usage dependencies are typically represented by a dashed arrowed line pointing from the client class to the supplier class.
- The arrow indicates the direction of the dependency, showing that the client class depends on the services provided by the supplier class.

An Example Class Diagram for a Business



Another Example Class Diagram



CRC Cards

- They belong in AGILE software development.
- Why did I put them After Class Diagrams?

Entity Class Using CRC Analysis

Class Name :BankAccount	
Responsiblities Maintain Balance Candidate Class	Collaborators Bank Other classes that may Collaborate with BankAccount Class to realize the responsibility

Behavioural UML Diagrams / Behaviour Diagrams

Behavior diagrams emphasize the behavior of a system by showing collaborations among objects and changes to the internal states of objects. They are used to describe the functionality of a system.

- Activity diagram
- Communication diagram
- Interaction overview diagram
- Sequence diagram
- State diagram
- Timing diagram
- Use case diagram

Q & A